

ADR: Amazon QuickSight Multi-Account Deployment Strategy for Kaizen

Context

Kaizen intends to deploy standardized Amazon QuickSight dashboards across 200 AWS accounts to ensure consistent layout, themes, and functionality across all client and subsidiary environments. Each account will visualize only its own data, enforcing data sovereignty. The solution must address scalability, security, user access, cost efficiency, and seamless dashboard embedding into client applications.

Challenges

- **Data Sovereignty:** Datasets must reside within client accounts.
- **Administrative Overhead:** No native CloudFormation support for QuickSight installation/user provisioning.
- **Embedding Dashboards:** Needs to offer an intuitive, seamless UX.
- **Cost Efficiency:** 1,000 Readers (5/account) and 3 Authors total—cost to be managed.
- **Scalability:** Dashboards need consistent and scalable multi-account deployment.
- **AWS Quotas & IAM Role Setup:** QuickSight allows only one dataset ingestion at a time per AWS account. However, during large-scale, parallel asset bundle imports across multiple accounts—especially when using shared regions, assumed roles, or centralized deployment tools—undocumented rate limits or throttling may occur. To ensure stable deployment, implement per-account ingestion-serialization logic (e.g., `await_running_ingestion`). Also, QuickSight's IAM service role must be manually configured via first login; no API is available to automate this step.

Architecture Schematic

See attached image: `image-20241119-095715.png`

Decision: Dashboard Template Management with Asset Bundles

Centralized Template Creation

- Create baseline dashboards, themes, and analyses in a source account.
- Export these assets using `StartAssetBundleExportJob`.
- Store exported JSON files in versioned S3 and push to GitHub for config tracking.

Target Account Setup

- Enable QuickSight and configure Admin/Reader roles.
- Create S3 bucket per target account for storing imported asset bundles.
- IAM roles must allow QuickSight and S3 access.
- Manual login required to trigger initial QuickSight service role creation due to lack of API support.

Client Dataset Preparation

- Store client data in S3, partitioned or prefix-based.
- Use AWS Glue crawlers to catalog datasets.
- Query using Athena.
- Apply IAM-based dataset access controls.
- Use SPICE for performance improvement when needed.

Asset Deployment

- Use `StartAssetBundleImportJob` in target accounts.
- Replace dataset ARNs as needed per client.
- Deploy with CloudFormation custom resource to trigger dashboard creation.
- Validate dataset mappings.
- Implement ingestion throttling logic (e.g., `await_running_ingestion`) to handle AWS limitation of one ingestion at a time.

Embedding Dashboards

- Use `GenerateEmbedUrlForRegisteredUser` for secure registered-user embedding.
- Avoid using `GetDashboardEmbedUrl` if embedding fails due to CSP/CORS issues.
- Configure CSP and CORS for secure embedding.

User Management

- 3 centralized Authors maintain dashboard content.
- Each of the 200 accounts has 5 Readers with secure access.

Security

- Apply strict IAM and encryption (S3, Athena, Glue).
- Datasets remain local to maintain sovereignty.
- Decryption via IAM permissions.

Automation

- Script user provisioning and subscriptions.
- Automate export/import of asset bundles.
- Use parameterized CloudFormation for account-specific setups.
- Align promotion strategy across dev/test/prod environments; asset bundle promotion responsibilities must be explicitly defined.

Performance (SPICE)

- Benefits:
 - In-memory speed
 - Cached results
 - Constraints:
 - Initial import latency
 - SPICE memory limits
 - Join/query complexity overhead
 - Single ingestion limit across datasets; parallel imports not supported
-

Governance, Observability & Monitoring

- **Versioning:** Use S3 + GitHub to manage asset bundles.
- **Monitoring:** Use CloudWatch + QuickSight usage metrics.
- **Auditing:** Review IAM, SPICE usage, dataset mappings regularly.
- **Automation:** Use SNS + CloudWatch for anomaly alerts.

SPICE Capacity Limits - Up to 1B rows or 1 TB per dataset in Enterprise Edition.

Cost Management

- **Licensing:**
- 3 Authors + 1,000 Readers = ~\$232,800 annually.
- **SPICE:**
- Authors: 10 GB free/month
- Readers: \$3/user/month + \$0.38/GB SPICE

SPICE (GB)	Volume (MM rows)	Monthly (\$)	Annual (\$)
5	1	1.9	22.8
10	5	3.8	45.6
20	10	7.6	91.2
50	25	19	228
100	50	38	456

Costs exclude Amazon Q, Capacity Pricing, Anonymous Access.

Consequences

Positive Outcomes

- **Security:** Strong IAM and encryption, no cross-account access.
- **Governance:** GitHub-backed version control.
- **Scalability:** Template-based, repeatable deployment.
- **Efficiency:** Automation minimizes manual work.
- **UX:** Embedded dashboards streamline access.
- **Risk Reduction:** Phased rollouts support progressive deployment.

Trade-offs

- **Admin Overhead:** Manual setup still required (e.g., enabling QuickSight, service role setup).
 - **Dev Effort:** Requires ongoing maintenance of scripts and integrations.
 - **Frontend Constraints:** IFrame-based embedding limits UI customization.
 - **NFR Clarity:** Early development missteps can occur without clear non-functional requirements and promotion strategy.
-

Alternatives Considered

1. Cross-Account IAM Role

2. *Pros*: Admins only in source account; secure.

3. *Cons*: High setup complexity.

4. Centralized QuickSight Account

5. *Pros*: Simple user management.

6. *Cons*: Breaks data sovereignty; hard to scale.

7. Manual Setup

8. *Pros*: No dev effort.

9. *Cons*: Not scalable, error-prone.

10. Custom Automation Scripts

11. *Pros*: Balanced flexibility.

12. *Cons*: Requires development and support.

Related Decisions - To be updated.

Reference Sources - <https://aws.amazon.com/solutions/guidance/multi-account-environments-on-amazon-quicksight/> - <https://aws.amazon.com/blogs/business-intelligence/automate-and-accelerate-your-amazon-quicksight-asset-deployments-using-the-new-apis/> - <https://aws.amazon.com/blogs/big-data/migrate-amazon-quicksight-across-aws-accounts/>

Votes | Name | Vote | Comments | |-----|-----|-----| | | | | | | |